ASSIGNMENT 8

NAME: Rohan Garg

SUBGROUP: 2023

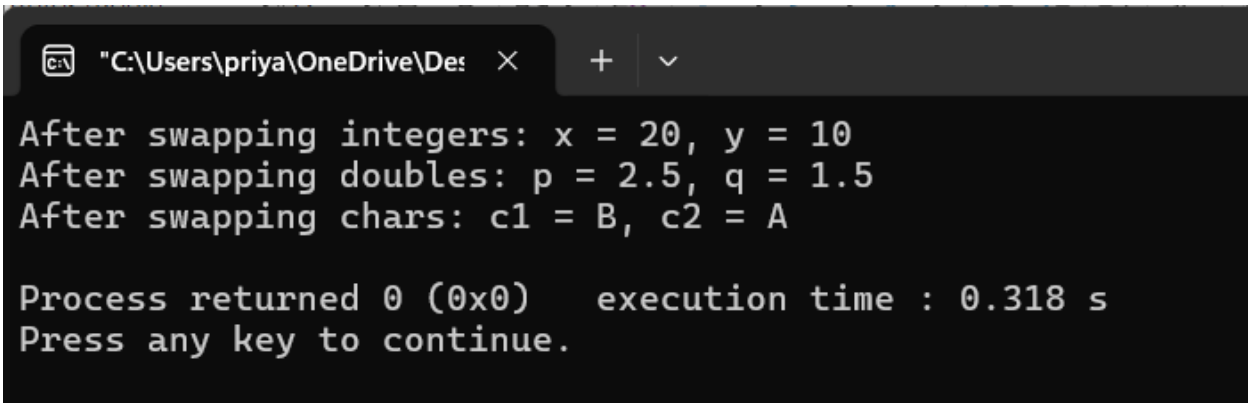
ROLL NO.: 1024150187

1. Write a function template to swap two variables of any data type.

CODE:

```
 #include <iostream>
using namespace std;

// Function template
template <typename T>
void swapValues(T &a, T &b) {
    T temp = a;
    a = b;
    b = temp;
}

int main() {
    int x = 10, y = 20;
    swapValues(x, y);
    cout << "After swapping integers: x = " << x << ", y = " << y << endl;

    double p = 1.5, q = 2.5;
    swapValues(p, q);
    cout << "After swapping doubles: p = " << p << ", q = " << q << endl;

    char c1 = 'A', c2 = 'B';
    swapValues(c1, c2);
    cout << "After swapping chars: c1 = " << c1 << ", c2 = " << c2 << endl;
```

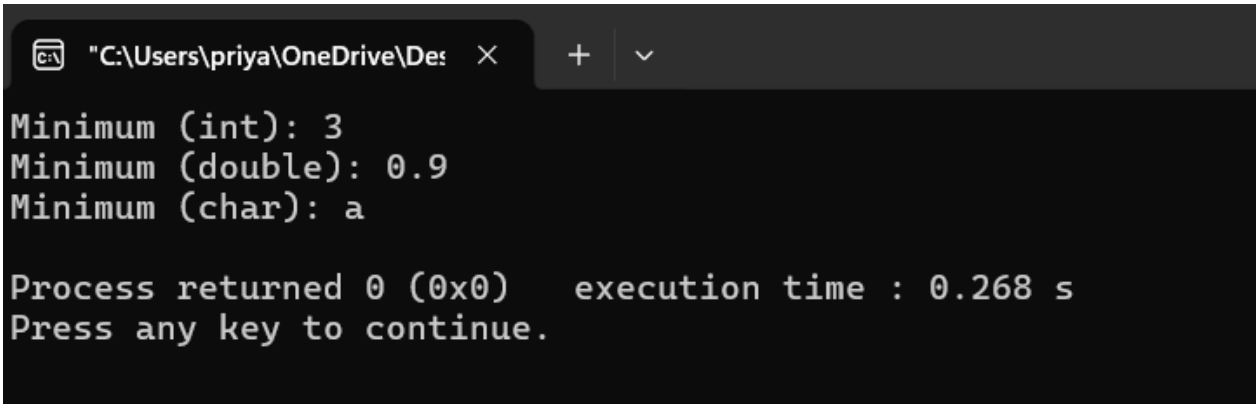
```
    return 0;
}
```



2. Write a function template to find the minimum element in an array.

```
#include <iostream>
using namespace std;

// Function template to find minimum element
template <typename T>
T findMin(T arr[], int size) {
    T min = arr[0];  // assume first element is minimum

    for (int i = 1; i < size; i++) {
        if (arr[i] < min) {
            min = arr[i];
        }
    }

    return min;
}
```

```
int main() {
   int arr1[] = {10, 5, 20, 3, 8};
   cout << "Minimum (int): " << findMin(arr1, 5) << endl;

   double arr2[] = {2.5, 1.1, 3.7, 0.9};
   cout << "Minimum (double): " << findMin(arr2, 4) << endl;

   char arr3[] = {'z', 'a', 'm', 'b'};
   cout << "Minimum (char): " << findMin(arr3, 4) << endl;

   return 0;
}
```

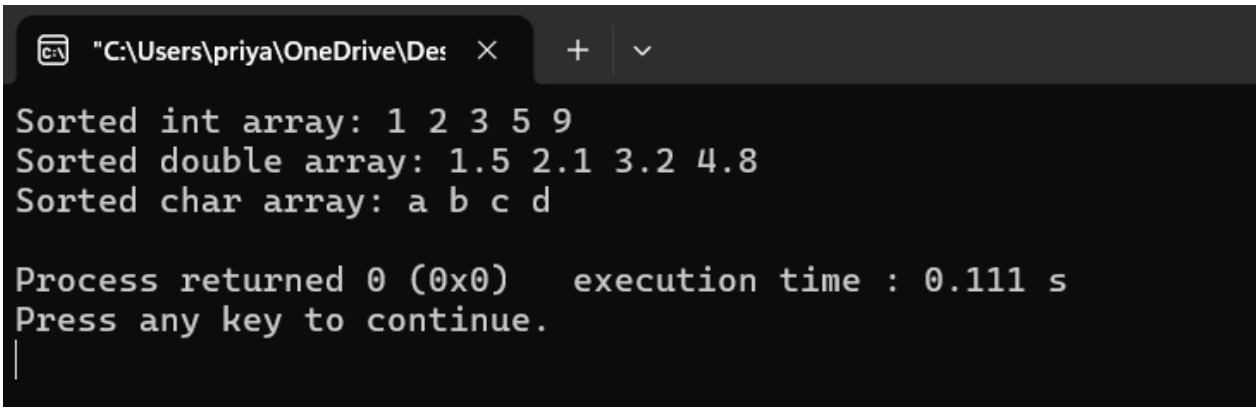


3. Write a function template to sort an array using bubble sort.

```
#include <iostream>
using namespace std;

// Function template for Bubble Sort
template <typename T>
void bubbleSort(T arr[], int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) {
```

```
            if (arr[j] > arr[j + 1]) {
                // swap
                T temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

// Function to print array
template <typename T>
void printArray(T arr[], int size) {
    for (int i = 0; i < size; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int main() {
    int arr1[] = {5, 2, 9, 1, 3};
    bubbleSort(arr1, 5);
    cout << "Sorted int array: ";
    printArray(arr1, 5);

    double arr2[] = {3.2, 1.5, 4.8, 2.1};
    bubbleSort(arr2, 4);
    cout << "Sorted double array: ";
    printArray(arr2, 4);
```

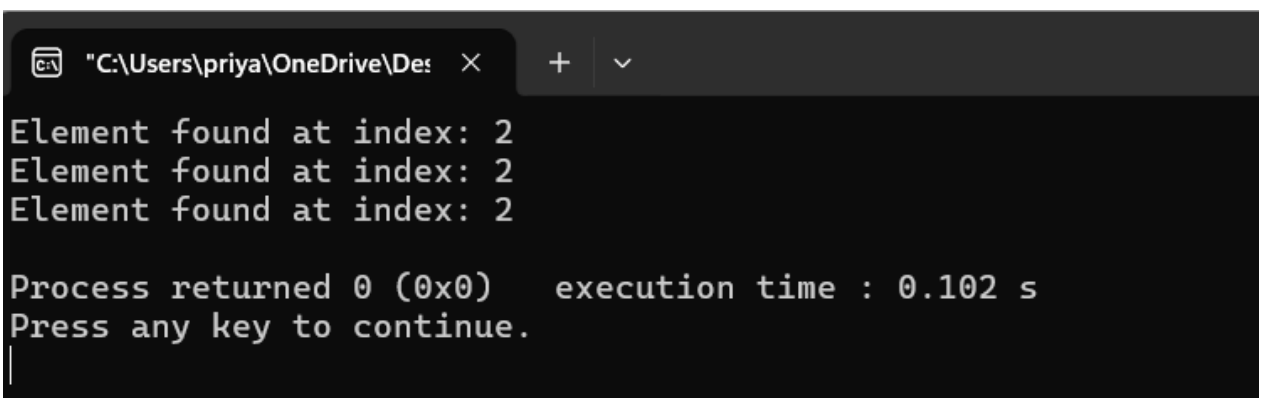
```
    char arr3[] = {'d', 'a', 'c', 'b'};
    bubbleSort(arr3, 4);
    cout << "Sorted char array: ";
    printArray(arr3, 4);

    return 0;
}
```



4. Write a function template to perform linear search.

```
#include <iostream>
using namespace std;

// Function template for Linear Search
template <typename T>
int linearSearch(T arr[], int size, T key) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == key) {
            return i;   // return index if found
        }
    }
    return -1;  // return -1 if not found
}
```

```
int main() {
    int arr1[] = {10, 20, 30, 40, 50};
    int index1 = linearSearch(arr1, 5, 30);
    cout << "Element found at index: " << index1 << endl;

    double arr2[] = {1.1, 2.2, 3.3, 4.4};
    int index2 = linearSearch(arr2, 4, 3.3);
    cout << "Element found at index: " << index2 << endl;

    char arr3[] = {'a', 'b', 'c', 'd'};
    int index3 = linearSearch(arr3, 4, 'c');
    cout << "Element found at index: " << index3 << endl;

    return 0;
}
```



5. Design overloaded versions of a function template process() such that:

   i) Accepts a single parameter

   ii) Accepts two parameters of the same type

   iii) Accepts two parameters of different types

```
#include <iostream>
using namespace std;
```

```
// i) Single parameter
template <typename T>
void process(T a) {
    cout << "Single parameter: " << a << endl;
}

// ii) Two parameters of same type
template <typename T>
void process(T a, T b) {
    cout << "Two parameters (same type): " << a << ", " << b << endl;
}

// iii) Two parameters of different types
template <typename T1, typename T2>
void process(T1 a, T2 b) {
    cout << "Two parameters (different types): " << a << ", " << b << endl;
}

int main() {
    process(10);            // single parameter
    process(5, 15);         // same type (int, int)
    process(3.5, 2.1);      // same type (double, double)
    process(10, 3.14);      // different types (int, double)
    process('A', 100);      // different types (char, int)

    return 0;
}
```

```
"C:\Users\priya\OneDrive\Des
Single parameter: 10
Two parameters (same type): 5, 15
Two parameters (same type): 3.5, 2.1
Two parameters (different types): 10, 3.14
Two parameters (different types): A, 100

Process returned 0 (0x0)   execution time : 0.102 s
Press any key to continue.
```

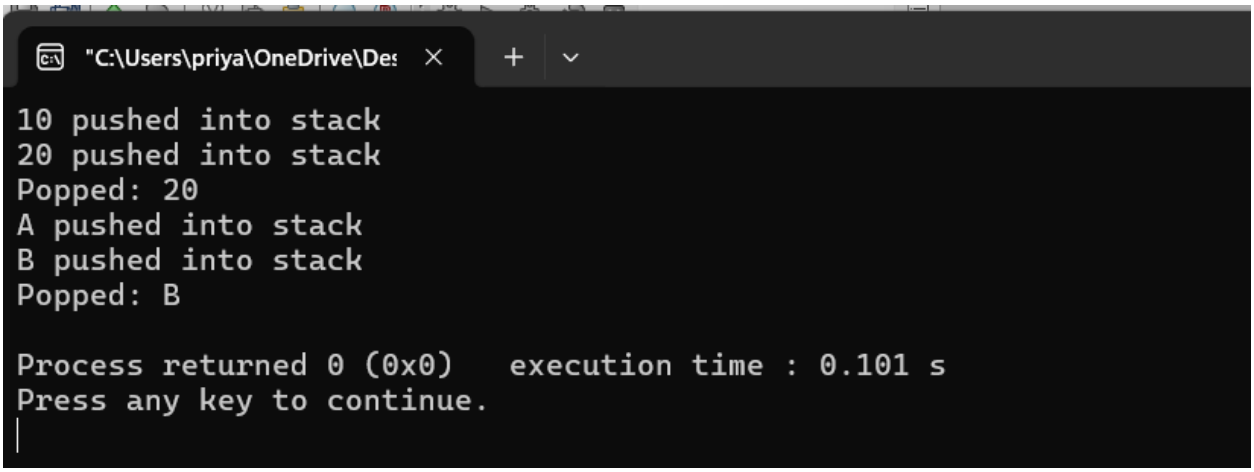
CLASS TEMPLATE QUESTIONS

1. Develop a class template for stack with push and pop operations.

```
#include <iostream>
using namespace std;

#define MAX 100   // maximum size of stack

// Class template
template <typename T>
class Stack {
private:
    T arr[MAX];
    int top;

public:
    // Constructor
    Stack() {
        top = -1;
    }

    // Push operation
```

```
    void push(T value) {
        if (top == MAX - 1) {
            cout << "Stack Overflow" << endl;
            return;
        }
        arr[++top] = value;
        cout << value << " pushed into stack" << endl;
    }

    // Pop operation
    T pop() {
        if (top == -1) {
            cout << "Stack Underflow" << endl;
            return T();   // return default value
        }
        return arr[top--];
    }

    // Display top element
    T peek() {
        if (top == -1) {
            cout << "Stack is empty" << endl;
            return T();
        }
        return arr[top];
    }
};

int main() {
    Stack<int> s1;
```

```
    s1.push(10);
    s1.push(20);
    cout << "Popped: " << s1.pop() << endl;

    Stack<char> s2;
    s2.push('A');
    s2.push('B');
    cout << "Popped: " << s2.pop() << endl;

    return 0;
}
```

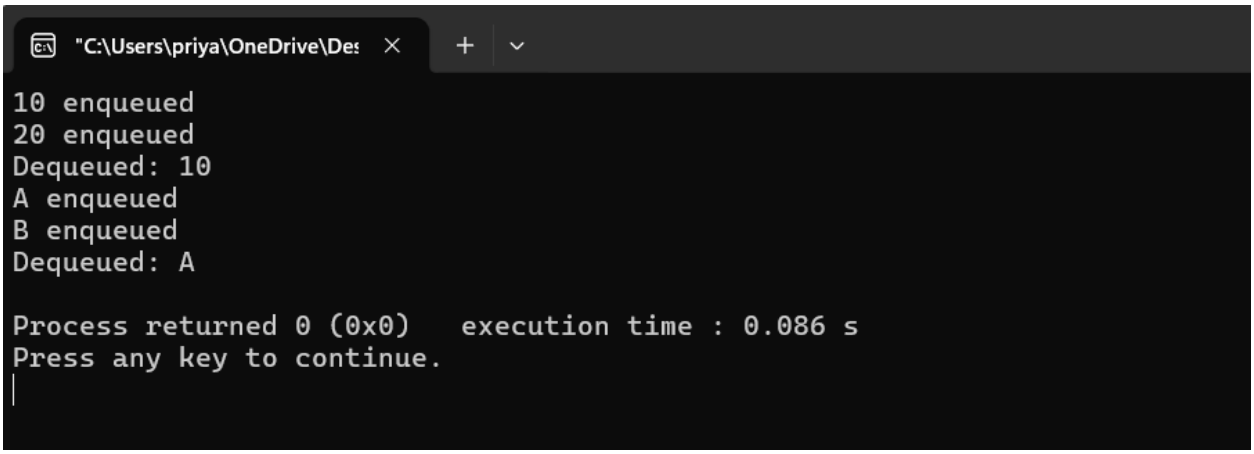


2. Develop a class template for queue with enqueue and dequeue operations.

```
#include <iostream>
using namespace std;

#define MAX 100   // maximum size of queue

// Class template
template <typename T>
```

```
class Queue {
private:
    T arr[MAX];
    int front, rear;

public:
    // Constructor
    Queue() {
        front = -1;
        rear = -1;
    }

    // Enqueue operation (insert)
    void enqueue(T value) {
        if (rear == MAX - 1) {
            cout << "Queue Overflow" << endl;
            return;
        }

        if (front == -1)  // first element
            front = 0;

        arr[++rear] = value;
        cout << value << " enqueued" << endl;
    }

    // Dequeue operation (remove)
    T dequeue() {
        if (front == -1 || front > rear) {
            cout << "Queue Underflow" << endl;
```

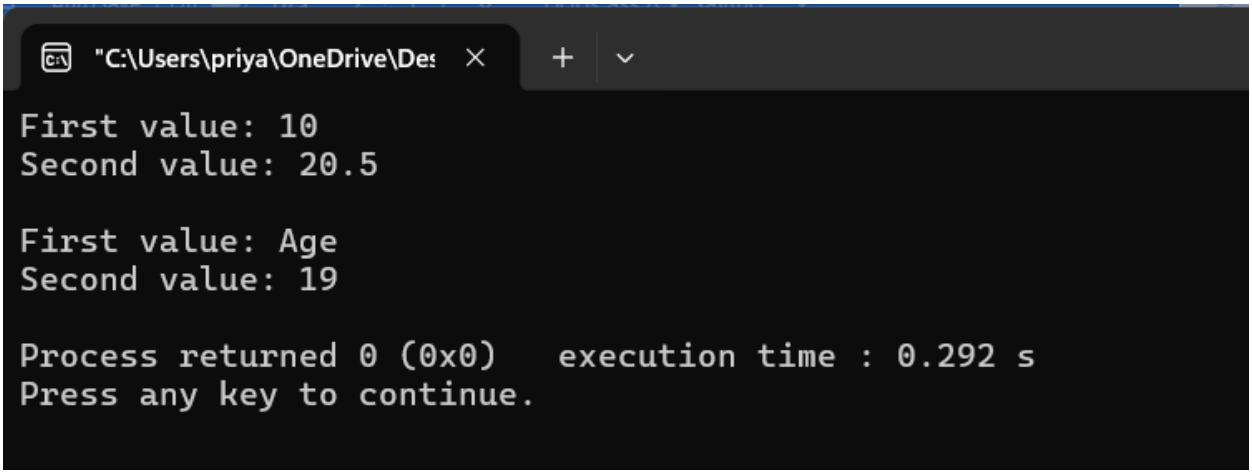
```
            return T();   // default value
        }

        return arr[front++];
    }

    // Display front element
    T peek() {
        if (front == -1 || front > rear) {
            cout << "Queue is empty" << endl;
            return T();
        }
        return arr[front];
    }
};

int main() {
    Queue<int> q1;
    q1.enqueue(10);
    q1.enqueue(20);
    cout << "Dequeued: " << q1.dequeue() << endl;

    Queue<char> q2;
    q2.enqueue('A');
    q2.enqueue('B');
    cout << "Dequeued: " << q2.dequeue() << endl;

    return 0;
}
```



3. Create a class template to store and display a pair of values.

```
#include <iostream>
using namespace std;

// Class template
template <class T1, class T2>
class Pair {
private:
    T1 first;
    T2 second;

public:
    // Constructor
    Pair(T1 a, T2 b) {
        first = a;
        second = b;
    }

    // Function to display values
```

```
    void display() {
        cout << "First value: " << first << endl;
        cout << "Second value: " << second << endl;
    }
};

int main() {
    // Example with int and float
    Pair<int, float> p1(10, 20.5);
    p1.display();

    cout << endl;

    // Example with string and int
    Pair<string, int> p2("Age", 19);
    p2.display();

    return 0;
}
```

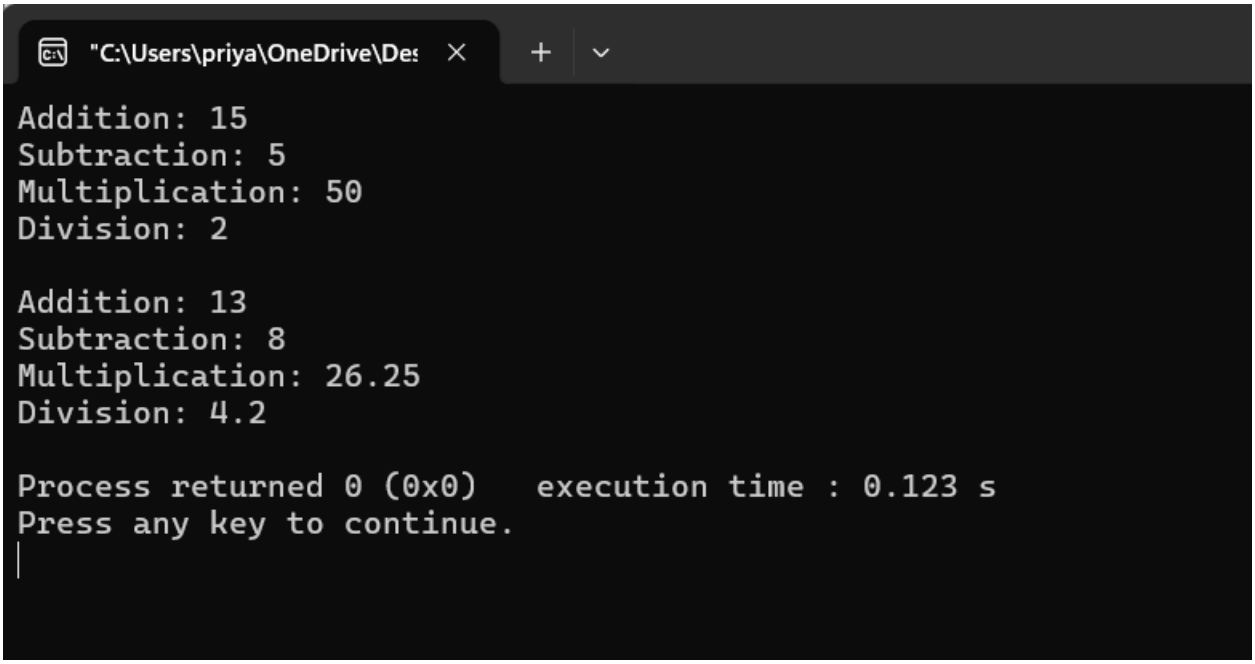


4. Create a class template for basic arithmetic operations.

```
#include <iostream>
using namespace std;

// Class template
template <class T>
class Arithmetic {
private:
    T a, b;

public:
    // Constructor
    Arithmetic(T x, T y) {
        a = x;
        b = y;
    }

    void add() {
        cout << "Addition: " << a + b << endl;
    }

    void subtract() {
        cout << "Subtraction: " << a - b << endl;
    }

    void multiply() {
```

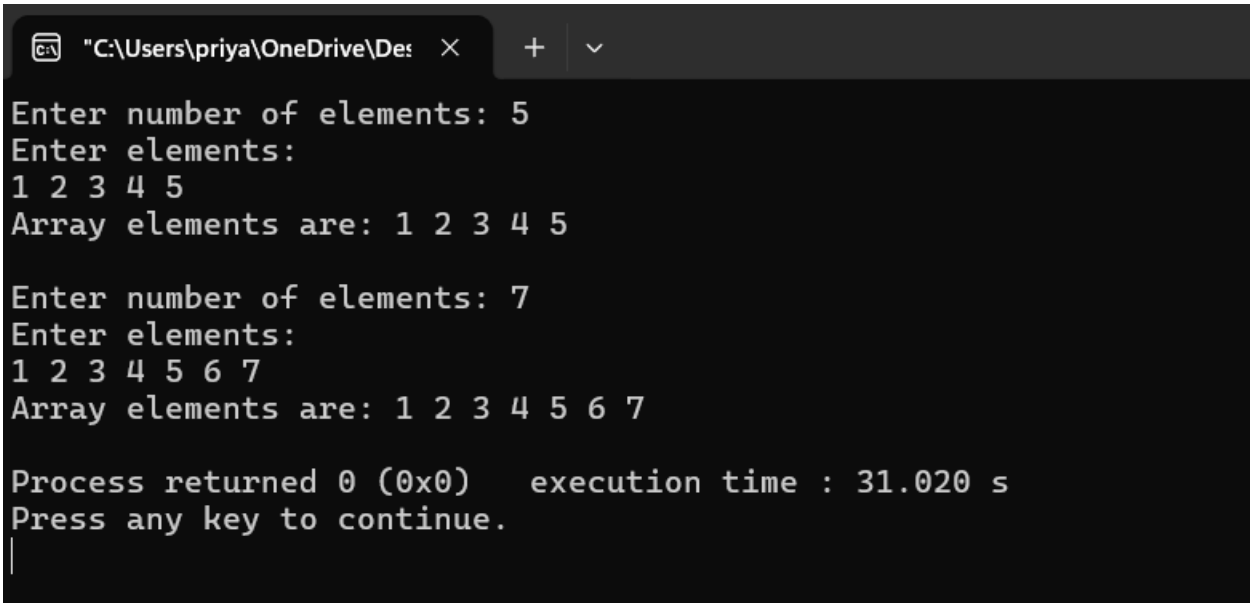
```
        cout << "Multiplication: " << a * b << endl;
    }

    void divide() {
        if (b != 0)
            cout << "Division: " << a / b << endl;
        else
            cout << "Division by zero not allowed!" << endl;
    }
};

int main() {
    // Example with integers
    Arithmetic<int> obj1(10, 5);
    obj1.add();
    obj1.subtract();
    obj1.multiply();
    obj1.divide();

    cout << endl;

    // Example with float
    Arithmetic<float> obj2(10.5, 2.5);
    obj2.add();
    obj2.subtract();
    obj2.multiply();
    obj2.divide();
```

```
    return 0;
}
```



5. Create a class template to input and display array elements.

```
#include <iostream>
using namespace std;

// Class template
template <class T>
class Array {
private:
    T arr[100];  // fixed size array
    int size;

public:
    // Function to input elements
```

```
    void input() {
        cout << "Enter number of elements: ";
        cin >> size;

        cout << "Enter elements:\n";
        for (int i = 0; i < size; i++) {
            cin >> arr[i];
        }
    }

    // Function to display elements
    void display() {
        cout << "Array elements are: ";
        for (int i = 0; i < size; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
};

int main() {
    // Example with integers
    Array<int> intArr;
    intArr.input();
    intArr.display();

    cout << endl;
```

```
    // Example with float
    Array<float> floatArr;
    floatArr.input();
    floatArr.display();

    return 0;
}
```